SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I B Priyanka (192421206) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Answer:

State representation: (x, y) where x = water in the 4-gallon jug (0–4) and y = water in the 3-gallon jug (0–3). Start state (0,0); goal state (2, n) – i.e. x = 2.

Production rules (operators):

- Fill 4-gal jug: (x,y), $x<4 \rightarrow (4,y)$
- Fill 3-gal jug: (x,y), $y<3 \rightarrow (x,3)$
- Empty 4-gal jug on ground: (x,y), $x>0 \rightarrow (0,y)$
- Empty 3-gal jug on ground: (x,y), $y>0 \rightarrow (x,0)$
- Pour 3-gal into 4-gal until 4-gal is full: (x,y), $x+y\geq4$, $y>0 \rightarrow (4, y-(4-x))$
- Pour 4-gal into 3-gal until 3-gal is full: (x,y), $x+y\geq3$, $x>0 \rightarrow (x-(3-y), 3)$
- Pour all of 3-gal into 4-gal: (x,y), $x+y\leq4$, $y>0 \rightarrow (x+y, 0)$
- Pour all of 4-gal into 3-gal: (x,y), $x+y\leq3$, $x>0 \rightarrow (0, x+y)$

Solution (shortest – 6 moves):

Step	Rule applied	State (4-gal, 3-gal)
0	Start	(0, 0)
1	Fill the 4-gallon jug from the pump	(4, 0)
2	Pour 4-gal into 3-gal until 3-gal is full	(1, 3)
3	Empty the 3-gallon jug on the ground	(1, 0)
4	Pour remaining 1 gallon from 4-gal into 3-gal	(0, 1)
5	Fill the 4-gallon jug again	(4, 1)
6	Pour 4-gal into 3-gal until full (only 2 gallons fit)	(2, 3) ← GOAL

Result: Exactly 2 gallons of water are in the 4-gallon jug after 6 moves. (An alternative 6-move solution is (0,0)→(0,3)→(3,0)→(3,3)→(4,2)→(0,2)→(2,0), which also ends with 2 gallons in the 4-gallon jug.)

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i. What are the percept's for this agent?

Ans: Percepts (inputs from sensors) of the Mars Rover:

- Camera images/video of terrain, rocks and soil (stereo vision, panoramic cameras)
- Terrain slope, tilt and wheel-slip (inclinometer, IMU, odometry)
- Distance to obstacles (lidar/hazard cameras), craters and cliffs
- Soil/rock composition data (spectrometer, X-ray/APXS, chemical and mineral sensors)
- Temperature, wind, dust, radiation and light level (weather/environment sensors)
- Battery/solar-power level, motor currents, internal temperature (health sensors)
- GPS-like position (visual odometry, star/sun tracker) and commands from Earth

ii. Characterize the operating environment.

Ans: Environment characteristics (PEAS – Environment):

Property	**Mars environment**
Observability	Partially observable (limited sensor range, dust, shadows)
Agents	Single agent (mostly; other rovers/orbiters are far away)
Determinism	Stochastic – wheel slip, sand, dust storms, rock movement cause uncertain outcomes
Episodic / Sequential	Sequential – current moves affect later exploration and power
Static / Dynamic	Dynamic – weather, dust storms, temperature and light change
Discrete / Continuous	Continuous – positions, speeds, time, sensor values
Known / Unknown	Partly unknown – terrain is new and must be explored

iii. What are the actions the agent can take?

Ans: Actions (actuators):

- Move forward/backward, turn left/right, stop, climb or avoid obstacles (wheels, steering)
- Rotate/tilt camera and antennas; adjust solar panels
- Extend robotic arm, drill, scoop or grind rock/soil to collect samples
- Place sample in on-board lab instruments and analyse it
- Take photos, store data and transmit results to Earth/orbiter
- Switch to sleep/power-saving mode; recharge; self-heat; send alerts

iv. How can one evaluate the performance of the agent?

Ans: Performance measure:

- Number and scientific value of samples correctly collected and analysed
- Area explored / distance covered safely
- Accuracy and quality of the data sent back to Earth
- Safety – no collisions, falls, stuck wheels or damage to the rover

- Energy and time efficiency; mission lifetime (rover stays operational)
- Fewer commands needed from Earth (degree of autonomy)

v. What sort of agent architecture do you think is most suitable for this agent? Why?
Ans: A goal-based agent with utility-based (learning) elements – a hybrid / layered architecture is most suitable.
- Radio signals take 4–24 minutes to reach Mars, so the rover cannot be remote-controlled in real time – it must decide by itself.
- A simple reflex agent is not enough: the environment is partially observable and actions have long-term effects (power, reaching a target rock).
- Goal-based: it plans paths to reach target sample sites and perform experiments.
- Utility-based: it trades off science value vs. risk, energy and time (e.g. choose a safer but longer path).
- Reflex layer for emergencies (stop at cliff edge, avoid collision) and a learning component to improve terrain navigation.
- It keeps an internal model (map and state) to cope with partial observability.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
Answer:
Problem formulation (incremental formulation):

Component	8-Queens definition
States	Any arrangement of 0 to 8 queens on the board (best: one queen per column, placed in the leftmost columns with no queen attacking another)
Initial state	Empty board – no queen placed
Actions	Add a queen to the next (leftmost empty) column in a row where it is not attacked
Transition model	Returns the board with the new queen added in the chosen square
Goal test	8 queens are on the board and none attacks another (no two share a row, column or diagonal)
Path cost	Not important – only the final configuration matters (each action costs 1)

Search-space size:
- Complete-state formulation (all 8 queens on board, move them): 64×63×…×57 ≈ 1.8×10^14 sequences (≈ 4.4×10^9 distinct placements, C(64,8)).
- Incremental, one queen per column: 8^8 ≈ 16.7 million arrangements.

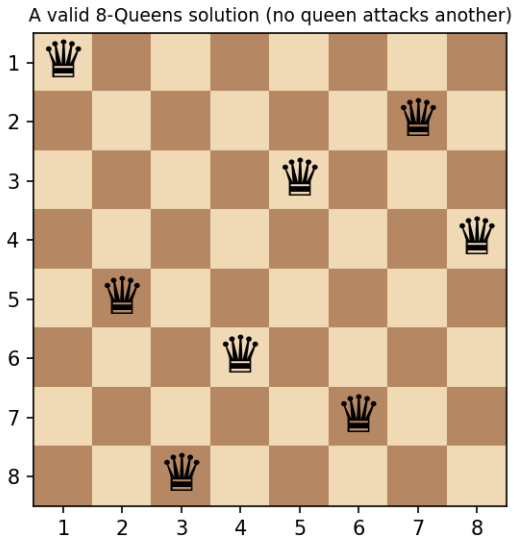
- Incremental, placing only in non-attacked squares: only 2,057 states – a huge reduction, so this formulation is preferred.

Search strategies:

- BFS: complete but stores a huge number of partial boards (high memory).
- DFS with backtracking: low memory (O(m)); place a queen column by column and backtrack when no safe row exists – the best uninformed method for this problem.
- Heuristic/local search (hill climbing, min-conflicts) can be used for very large n.

Why the figure in the question is not a solution: the queen in the first column and the queen in the last column lie on the same diagonal, so they attack each other – the goal test fails.

A valid solution found by DFS/backtracking (row positions for columns 1–8 = 1, 5, 8, 6, 3, 7, 2, 4):



Checking: all rows are different, all columns are different and no two queens share a diagonal, so the goal test is satisfied. (The puzzle has 92 solutions, 12 distinct ones after removing rotations/reflections.)

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Answer:

Type of agent: A **goal-based problem-solving agent** (with a utility-based choice of cab type). The goal is to reach the destination; the agent formulates the problem, searches for a sequence of actions (booking → pickup → ride → drop) and executes it. Because the cab types (mini, micro, sedan, shared, prime…) differ in price, time and comfort, the agent picks the one with the best utility for the customer.

Problem formulation:

- Initial state: customer at pickup location, app opened, nothing booked
- Actions: select cab type, confirm booking, wait for driver, ride, pay
- Goal state: customer reached the destination
- Transition model: each action changes the booking/travel status
- Path cost / utility: fare + waiting time + travel time + comfort preference

Pseudocode:

```
function CAB-BOOKING-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty),
              state, goal, problem

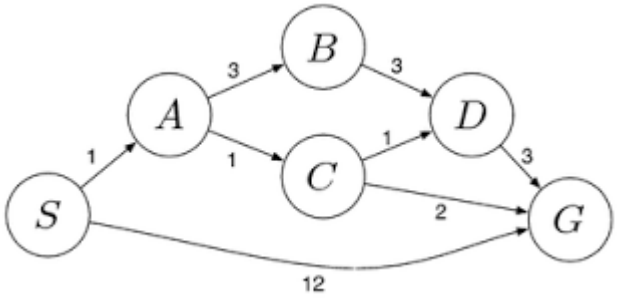
  state <- UPDATE-STATE(state, percept)   // pickup, drop, time, traffic,
cab availability
  if seq is empty then
      goal    <- FORMULATE-GOAL(state)    // reach destination from pickup
location
      problem <- FORMULATE-PROBLEM(state, goal)
      seq     <- SEARCH(problem)          // booking plan
      if seq = failure then return NO-CAB-AVAILABLE
  action <- FIRST(seq)
  seq    <- REST(seq)
  return action

function SEARCH(problem) returns plan
  options <- GET-AVAILABLE-CABS(pickup, drop)    // mini, micro, sedan,
shared, prime ...
  best <- null; bestUtility <- -infinity
  for each cab in options do
      fare <- ESTIMATE-FARE(cab, distance)
      eta  <- ESTIMATE-ARRIVAL-TIME(cab)
      u     <- w1*comfort(cab) - w2*fare - w3*eta       // customer's
preference weights
      if u > bestUtility then best <- cab; bestUtility <- u
  if best = null then return failure
  return [SELECT(best), CONFIRM-BOOKING, WAIT-FOR-DRIVER, START-RIDE,
REACH-DESTINATION, PAY]
```

Result: The agent returns the best cab for the customer and the action sequence that takes him from the source to the destination.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Answer: Uniform Cost Search (UCS)

Graph edges (cost): S→A = 1, S→G = 12, A→B = 3, A→C = 1, B→D = 3, C→D = 1, C→G = 2, D→G = 3.

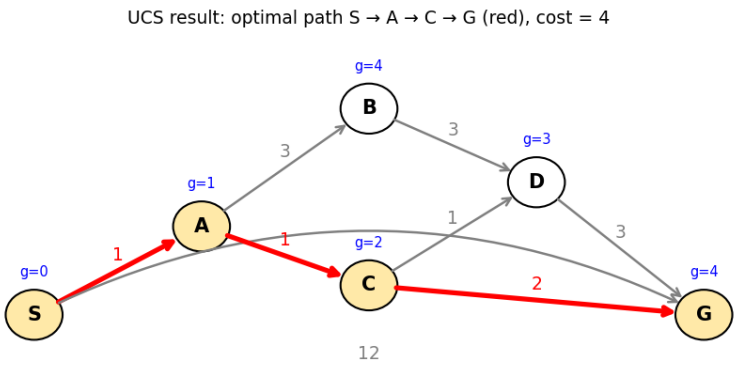
Method: UCS always expands the frontier node with the lowest path cost g(n) (priority queue). It is complete and optimal when step costs are positive.

Step-by-step trace:

Step	Node expanded	Frontier after expansion (node : g)	Explored
1	S (0)	A:1, G:12	S
2	A (1)	C:2, B:4, G:12	S, A
3	C (2)	D:3, B:4, G:4 (via C; 4 < 12 so G updated)	S, A, C
4	D (3)	B:4, G:4 (G via D = 6 > 4, not better)	S, A, C, D
5	B (4)	G:4 (D via B = 7 > 3, ignored)	S, A, C, D, B
6	G (4) – goal test passed	– (stop)	S, A, C, D, B, G

All paths from S to G:

Path	Total cost
S → A → C → G	1 + 1 + 2 = 4 (minimum)
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → G	12



Final answer: The least-cost path is **S → A → C → G with total cost = 4** (fuel + time units). Direct route S → G costs 12, so the company saves 8 units by using the route through warehouses A and C. Nodes expanded: S, A, C, D, B, G (order of increasing cost).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem